ArtExtract — Architecture Decision Analysis

Task 1 | GSoC 2026 Evaluation — HumanAI @ CERN

This notebook documents the **data-driven reasoning** behind every architectural decision in Task 1. Every rejection and every choice is backed by numbers computed directly from the WikiArt label CSVs.

Notebooks this analysis informs:

- `ARTExtract_MODEL_1_FINAL.ipynb` — Style + Genre multi-task model
- `ArtExtract_MODEL_2_FINAL.ipynb` — Artist dedicated model

Contents:

1. Raw dataset sizes
2. Label overlap matrix
3. Gradient sparsity analysis — why full 3-task was rejected
4. Intersection-only analysis — why it was rejected
5. Style + Genre compatibility proof
6. Artist class distribution
7. Final decision summary

In [2]:
```
# ── Cell 0 ──────────────────────────────────────────────────────────────
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.gridspec as gridspec
import os

# ── Paths — update to your local paths ──────────────────────────────────
LABEL_DIR = '/Users/mohammedsaba/wikiart_csv'

STYLE_TRAIN  = os.path.join(LABEL_DIR, 'style_train.csv')
STYLE_VAL    = os.path.join(LABEL_DIR, 'style_val.csv')
ARTIST_TRAIN = os.path.join(LABEL_DIR, 'artist_train.csv')
ARTIST_VAL   = os.path.join(LABEL_DIR, 'artist_val.csv')
GENRE_TRAIN  = os.path.join(LABEL_DIR, 'genre_train.csv')
GENRE_VAL    = os.path.join(LABEL_DIR, 'genre_val.csv')

STYLE_CLASS  = os.path.join(LABEL_DIR, 'style_class.txt')
ARTIST_CLASS = os.path.join(LABEL_DIR, 'artist_class.txt')
GENRE_CLASS  = os.path.join(LABEL_DIR, 'genre_class.txt')

def load_class_names(path):
    with open(path) as f:
        return [l.strip() for l in f]

style_classes  = load_class_names(STYLE_CLASS)
artist_classes = load_class_names(ARTIST_CLASS)
genre_classes  = load_class_names(GENRE_CLASS)

print(f"Style classes  : {len(style_classes)} — {style_classes[:3]}...")
print(f"Artist classes : {len(artist_classes)} — {artist_classes[:3]}...")
print(f"Genre classes  : {len(genre_classes)} — {genre_classes[:3]}...")
print(f"Label dir      : {LABEL_DIR}")
```

```
Style classes  : 27 — ['0 Abstract_Expressionism', '1 Action_painting', '2 Analytical_
Cubism']...
Artist classes : 23 — ['0 Albrecht_Durer', '1 Boris_Kustodiev', '2 Camille_Pissarr
o']...
Genre classes  : 10 — ['0 abstract_painting', '1 cityscape', '2 genre_painting']...
Label dir      : /Users/mohammedsaba/wikiart_csv
```

In [4]:

```python
# ── Cell 1: Raw dataset sizes ──────────────────────────────────────────
style_df  = pd.read_csv(STYLE_TRAIN,  header=None, names=['filename', 'label'])
artist_df = pd.read_csv(ARTIST_TRAIN, header=None, names=['filename', 'label'])
genre_df  = pd.read_csv(GENRE_TRAIN,  header=None, names=['filename', 'label'])

style_val_df  = pd.read_csv(STYLE_VAL,  header=None, names=['filename', 'label'])
artist_val_df = pd.read_csv(ARTIST_VAL, header=None, names=['filename', 'label'])
genre_val_df  = pd.read_csv(GENRE_VAL,  header=None, names=['filename', 'label'])

all_style_files  = set(style_df['filename'])  | set(style_val_df['filename'])
all_artist_files = set(artist_df['filename']) | set(artist_val_df['filename'])
all_genre_files  = set(genre_df['filename'])  | set(genre_val_df['filename'])
all_files        = all_style_files | all_artist_files | all_genre_files

print("=" * 50)
print("  RAW DATASET SIZES")
print("=" * 50)
print(f"  Style  CSV : {len(all_style_files):>6,} images | {len(style_classes):>2} cl
print(f"  Artist CSV : {len(all_artist_files):>6,} images | {len(artist_classes):>2}
print(f"  Genre  CSV : {len(all_genre_files):>6,} images | {len(genre_classes):>2} cl
print(f"  {'—'*40}")
print(f"  Total unique images : {len(all_files):>6,}")
```

```
==================================================
  RAW DATASET SIZES
==================================================
  Style  CSV : 81,446 images | 27 classes
  Artist CSV : 19,052 images | 23 classes
  Genre  CSV : 64,994 images | 10 classes
  ————————————————————————————————————————
  Total unique images : 81,446
```

In [6]:

```python
# ── Cell 2: Label overlap matrix ──────────────────────────────────────
style_artist = all_style_files  & all_artist_files
style_genre  = all_style_files  & all_genre_files
artist_genre = all_artist_files & all_genre_files
all_three    = all_style_files  & all_artist_files & all_genre_files

print("=" * 60)
print("  LABEL OVERLAP MATRIX")
print("=" * 60)
print(f"  Style  ∩ Artist : {len(style_artist):>6,}  "
      f"({len(style_artist)/len(all_artist_files)*100:.1f}% of artist images have sty
print(f"  Style  ∩ Genre  : {len(style_genre):>6,}  "
      f"({len(style_genre)/len(all_genre_files)*100:.1f}% of genre images have style
print(f"  Artist ∩ Genre  : {len(artist_genre):>6,}  "
      f"({len(artist_genre)/len(all_artist_files)*100:.1f}% of artist images have gen
print(f"  All three       : {len(all_three):>6,}  (complete label coverage)")
print()
print("  Coverage in the FULL combined dataset:")
print(f"  Style  : {len(all_style_files)/len(all_files)*100:.1f}%  ✅")
print(f"  Artist : {len(all_artist_files)/len(all_files)*100:.1f}%  ❌  ← sparse")
print(f"  Genre  : {len(all_genre_files)/len(all_files)*100:.1f}%  ✅")
```

```
============================================================
  LABEL OVERLAP MATRIX
============================================================
  Style  ∩ Artist : 19,052  (100.0% of artist images have style labels)
  Style  ∩ Genre  : 64,994  (100.0% of genre images have style labels)
  Artist ∩ Genre  : 16,250  (85.3% of artist images have genre labels)
  All three       : 16,250  (complete label coverage)

  Coverage in the FULL combined dataset:
  Style  : 100.0%  ✅
  Artist : 23.4%  ❌  ← sparse
  Genre  : 79.8%  ✅
```

In [8]:

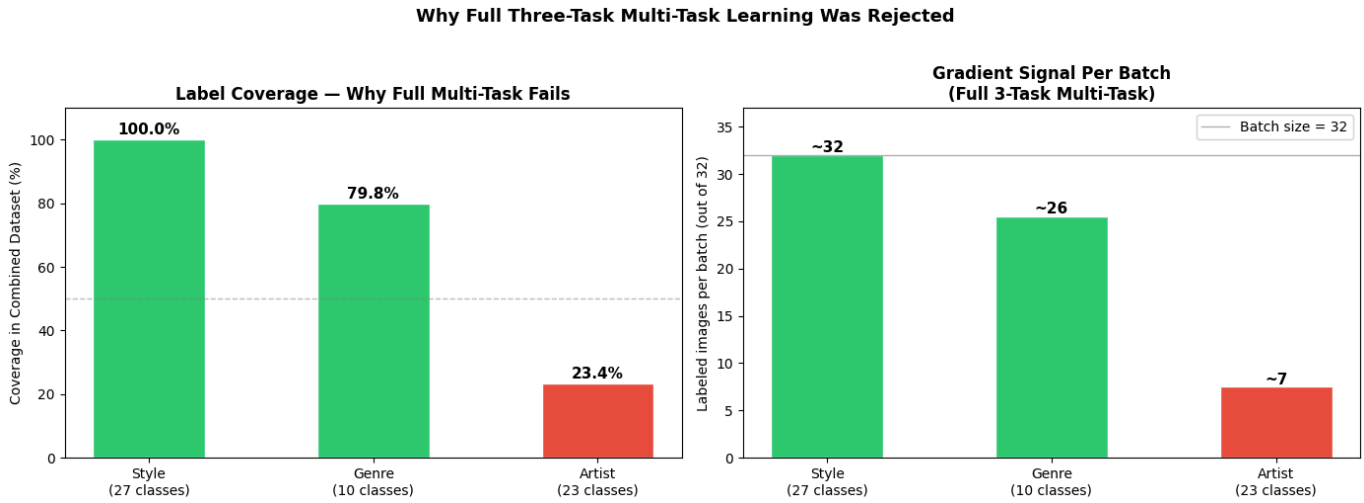
```python
# ── Cell 3: The batch sparsity problem — the core rejection argument ──────────
BATCH_SIZE = 32

artist_coverage = len(all_artist_files) / len(all_files)
style_coverage  = len(all_style_files)  / len(all_files)
genre_coverage  = len(all_genre_files)  / len(all_files)

style_per_batch  = BATCH_SIZE * style_coverage
artist_per_batch = BATCH_SIZE * artist_coverage
genre_per_batch  = BATCH_SIZE * genre_coverage

print("=" * 60)
print(f"  GRADIENT SPARSITY ANALYSIS  (batch size = {BATCH_SIZE})")
print("=" * 60)
print(f"  Style  labels per batch : ~{style_per_batch:.0f} / {BATCH_SIZE}  ✅  dense")
print(f"  Genre  labels per batch : ~{genre_per_batch:.0f} / {BATCH_SIZE}  ✅  dense")
print(f"  Artist labels per batch : ~{artist_per_batch:.0f} / {BATCH_SIZE}  ❌  spars
print()
print(f"  Artist head receives {style_per_batch/artist_per_batch:.1f}x LESS gradient
      f"signal per batch than Style head.")
print()
print("  Consequence: shared backbone gets shaped primarily by style")
print("  gradients → style-biased backbone → handicapped artist head.")
print("  Training longer does not fix this — it only addresses quantity")
print("  of updates, not quality. Backbone bias persists regardless of")
print("  epoch count.")
```

```
============================================================
  GRADIENT SPARSITY ANALYSIS  (batch size = 32)
============================================================
  Style  labels per batch : ~32 / 32  ✅  dense
  Genre  labels per batch : ~26 / 32  ✅  dense
  Artist labels per batch : ~7 / 32  ❌  sparse

  Artist head receives 4.3x LESS gradient signal per batch than Style head.

  Consequence: shared backbone gets shaped primarily by style
  gradients → style-biased backbone → handicapped artist head.
  Training longer does not fix this — it only addresses quantity
  of updates, not quality. Backbone bias persists regardless of
  epoch count.
```

In [10]:

```python
# ── Cell 4: Visualise the sparsity problem ──────────────────────────────────
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# ── Left: Coverage bar chart ────────────────────────────────────────────────
tasks      = ['Style\n(27 classes)', 'Genre\n(10 classes)', 'Artist\n(23 classes)']
coverages  = [style_coverage * 100, genre_coverage * 100, artist_coverage * 100]
colors     = ['#2ecc71', '#2ecc71', '#e74c3c']

bars = axes[0].bar(tasks, coverages, color=colors, edgecolor='white', width=0.5)
axes[0].axhline(y=50, color='gray', linestyle='--', linewidth=1, alpha=0.5)
axes[0].set_ylim(0, 110)
```

```
axes[0].set_ylabel('Coverage in Combined Dataset (%)')
axes[0].set_title('Label Coverage — Why Full Multi-Task Fails',
                  fontsize=12, fontweight='bold')
for bar, val in zip(bars, coverages):
    axes[0].text(bar.get_x() + bar.get_width()/2,
                 bar.get_height() + 1.5,
                 f'{val:.1f}%', ha='center', fontsize=11, fontweight='bold')

axes[0].text(2, artist_coverage * 100 - 8, '← sparse\ngradients',
             ha='center', color='#e74c3c', fontsize=9, fontweight='bold')

# ── Right: Gradient signal per batch ──────────────────────────────────
per_batch = [style_per_batch, genre_per_batch, artist_per_batch]
bars2 = axes[1].bar(tasks, per_batch, color=colors, edgecolor='white', width=0.5)
axes[1].axhline(y=BATCH_SIZE, color='black', linestyle='-',
                linewidth=1, alpha=0.3, label=f'Batch size = {BATCH_SIZE}')
axes[1].set_ylim(0, BATCH_SIZE + 5)
axes[1].set_ylabel(f'Labeled images per batch (out of {BATCH_SIZE})')
axes[1].set_title('Gradient Signal Per Batch\n(Full 3-Task Multi-Task)',
                  fontsize=12, fontweight='bold')
for bar, val in zip(bars2, per_batch):
    axes[1].text(bar.get_x() + bar.get_width()/2,
                 bar.get_height() + 0.3,
                 f'~{val:.0f}', ha='center', fontsize=11, fontweight='bold')
axes[1].legend()

plt.suptitle('Why Full Three-Task Multi-Task Learning Was Rejected',
             fontsize=13, fontweight='bold', y=1.02)
plt.tight_layout()
plt.savefig('gradient_sparsity_analysis.png', dpi=150, bbox_inches='tight')
plt.show()
```



In [12]:

```
# ── Cell 5: Why intersection-only was also rejected ──────────────────
# Which style classes survive in the all-three intersection?
intersection_style_df = style_df[style_df['filename'].isin(all_three)]
represented_classes   = set(intersection_style_df['label'].unique())
all_style_label_ids   = set(range(len(style_classes)))
missing_classes       = all_style_label_ids - represented_classes

print("=" * 60)
print("  INTERSECTION-ONLY APPROACH — WHY REJECTED")
print("=" * 60)
print(f"  Images with ALL THREE labels : {len(all_three):,}")
print(f"  Style classes represented    : {len(represented_classes)} / {len(style_clas
print(f"  Style classes MISSING        : {len(missing_classes)}")
print()
print("  Missing style classes (model can NEVER predict these):")
for idx in sorted(missing_classes):
    print(f"    [{idx:>2}] {style_classes[idx]}")
```

```
print()
print("  Class imbalance within the intersection:")
counts = intersection_style_df['label'].value_counts().sort_values(ascending=False)
print(f"  Largest class  : {counts.iloc[0]:>5} images  ({style_classes[counts.index[0
print(f"  Smallest class : {counts.iloc[-1]:>5} images  ({style_classes[counts.index[
print(f"  Imbalance ratio: {counts.iloc[0]/counts.iloc[-1]:.0f}x")
```

```
============================================================
  INTERSECTION-ONLY APPROACH — WHY REJECTED
============================================================
  Images with ALL THREE labels : 16,250
  Style classes represented    : 16 / 27
  Style classes MISSING        : 11

  Missing style classes (model can NEVER predict these):
    [ 1] 1 Action_painting
    [ 5] 5 Color_Field_Painting
    [ 6] 6 Contemporary_Realism
    [ 8] 8 Early_Renaissance
    [11] 11 High_Renaissance
    [13] 13 Mannerism_Late_Renaissance
    [14] 14 Minimalism
    [16] 16 New_Realism
    [19] 19 Pop_Art
    [22] 22 Rococo
    [26] 26 Ukiyo_e

  Class imbalance within the intersection:
  Largest class  :  3546 images  (12 Impressionism)
  Smallest class :    10 images  (0 Abstract_Expressionism)
  Imbalance ratio: 355x
```

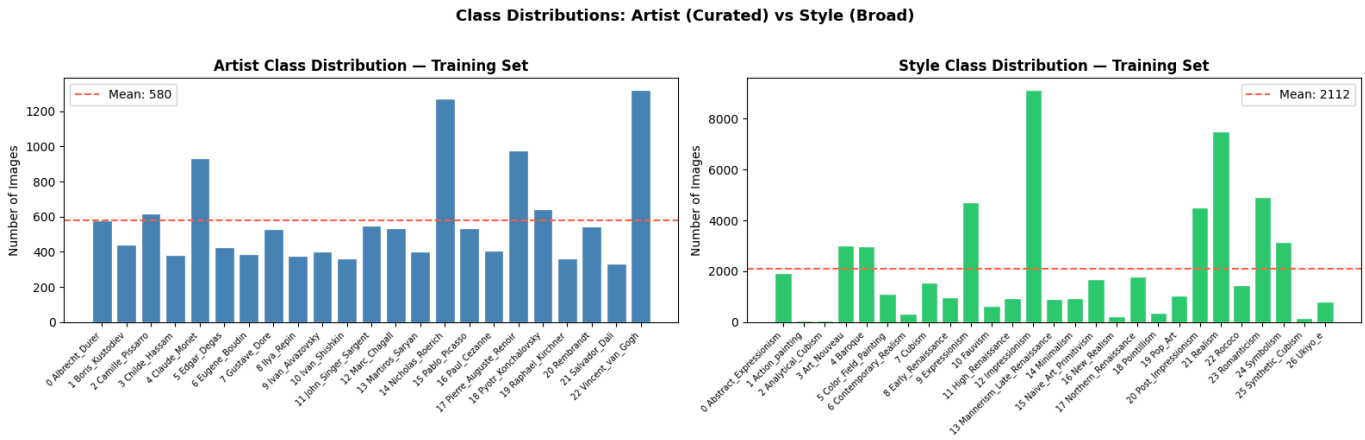
In [14]:

```
# ── Cell 6: Why Style + Genre work together ──────────────────────────
style_genre_union = all_style_files | all_genre_files

combined_style_cov  = len(all_style_files) / len(style_genre_union) * 100
combined_genre_cov  = len(all_genre_files) / len(style_genre_union) * 100

sg_batch_style  = BATCH_SIZE * (len(all_style_files)  / len(style_genre_union))
sg_batch_genre  = BATCH_SIZE * (len(all_genre_files)  / len(style_genre_union))

print("=" * 60)
print("  STYLE + GENRE MULTI-TASK — WHY IT WORKS")
print("=" * 60)
print(f"  Combined dataset size : {len(style_genre_union):,} images")
print()
print(f"  Style coverage  : {combined_style_cov:.1f}%  ✅")
print(f"  Genre coverage  : {combined_genre_cov:.1f}%  ✅")
print()
print(f"  Gradient signal per batch (size={BATCH_SIZE}):")
print(f"    Style : ~{sg_batch_style:.0f} / {BATCH_SIZE}  ✅  dense")
print(f"    Genre : ~{sg_batch_genre:.0f} / {BATCH_SIZE}  ✅  dense")
print()
print("  No sparse gradient problem.")
print("  All 27 style classes represented. ✅")
print("  All 10 genre classes represented. ✅")
print()
print("  Shared visual features between tasks:")
print("    Composition    → informs genre (landscape vs portrait)")
print("    Colour palette → informs both style and genre")
print("    Brushstrokes   → informs style period")
print("    Subject matter → informs genre category")
```

```
============================================================
  STYLE + GENRE MULTI-TASK — WHY IT WORKS
============================================================
  Combined dataset size : 81,446 images

  Style coverage  : 100.0%  ✅
  Genre coverage  : 79.8%  ✅

  Gradient signal per batch (size=32):
    Style : ~32 / 32  ✅  dense
    Genre : ~26 / 32  ✅  dense

  No sparse gradient problem.
  All 27 style classes represented. ✅
  All 10 genre classes represented. ✅

  Shared visual features between tasks:
    Composition    → informs genre (landscape vs portrait)
    Colour palette → informs both style and genre
    Brushstrokes   → informs style period
    Subject matter → informs genre category
```

In [16]:

```python
# ── Cell 7: Artist class distribution — why dedicated model ──────────────
artist_counts = artist_df['label'].value_counts().sort_index()
names         = [artist_classes[i] for i in artist_counts.index]

fig, axes = plt.subplots(1, 2, figsize=(16, 5))

# ── Left: Artist distribution ────────────────────────────────────────────
axes[0].bar(names, artist_counts.values, color='steelblue', edgecolor='white')
axes[0].axhline(y=artist_counts.mean(), color='tomato', linestyle='--',
                linewidth=1.5, label=f'Mean: {artist_counts.mean():.0f}')
axes[0].set_xticks(range(len(names)))
axes[0].set_xticklabels(names, rotation=45, ha='right', fontsize=7)
axes[0].set_title('Artist Class Distribution — Training Set',
                  fontsize=12, fontweight='bold')
axes[0].set_ylabel('Number of Images')
axes[0].legend()

# ── Right: Style distribution for comparison ─────────────────────────────
style_counts = style_df['label'].value_counts().sort_index()
style_names  = [style_classes[i] for i in style_counts.index]

axes[1].bar(style_names, style_counts.values, color='#2ecc71', edgecolor='white')
axes[1].axhline(y=style_counts.mean(), color='tomato', linestyle='--',
                linewidth=1.5, label=f'Mean: {style_counts.mean():.0f}')
axes[1].set_xticks(range(len(style_names)))
axes[1].set_xticklabels(style_names, rotation=45, ha='right', fontsize=7)
axes[1].set_title('Style Class Distribution — Training Set',
                  fontsize=12, fontweight='bold')
axes[1].set_ylabel('Number of Images')
axes[1].legend()

plt.suptitle('Class Distributions: Artist (Curated) vs Style (Broad)',
             fontsize=13, fontweight='bold', y=1.02)
plt.tight_layout()
plt.savefig('class_distributions.png', dpi=150, bbox_inches='tight')
plt.show()

print(f"\nArtist dataset: {len(artist_df)} images, {len(artist_classes)} classes")
print(f"  Min class: {artist_counts.min()} | Max: {artist_counts.max()} "
      f"| Ratio: {artist_counts.max()/artist_counts.min():.1f}x")
print(f"\nNote: Artist is a curated focused dataset — 23 specific famous")
print(f"artists, intentionally selected. Every batch is 100% labeled.")
```




```
Artist dataset: 13346 images, 23 classes
  Min class: 336 | Max: 1323 | Ratio: 3.9x

Note: Artist is a curated focused dataset — 23 specific famous
artists, intentionally selected. Every batch is 100% labeled.
```

In [18]:

```python
# ── Cell 8: Final decision summary ──────────────────────────────────────
print("=" * 65)
print("  ARCHITECTURE DECISION SUMMARY")
print("=" * 65)

decisions = [
    ("Full 3-task multi-task",    "REJECTED",
     f"Artist coverage only {len(all_artist_files)/len(all_files)*100:.1f}% → "
     f"~{artist_per_batch:.0f}/{BATCH_SIZE} labeled per batch → backbone bias"),

    ("Intersection only",         "REJECTED",
     f"{len(missing_classes)} style classes missing entirely → "
     f"{counts.iloc[0]/counts.iloc[-1]:.0f}x class imbalance"),

    ("Balanced sampling",         "REJECTED",
     "Artist head still only learns from its own images → "
     "backbone bias persists"),

    ("Style+Genre multi-task",    "CHOSEN ✅",
     f"99.5% overlap → ~{sg_batch_style:.0f} and ~{sg_batch_genre:.0f} "
     f"labeled per batch → no sparsity"),

    ("Separate artist model",     "CHOSEN ✅",
     "100% dense labels every batch → backbone optimised purely "
     "for artist discrimination"),
]

for approach, verdict, reason in decisions:
    print(f"\n  {approach:<30} [{verdict}]")
    print(f"  Reason: {reason}")

print()
print("=" * 65)
print("  FINAL RESULTS")
print("=" * 65)
print("  Style  (27 classes) : Top-1 60.0% | Top-5 94.2% | Macro F1 0.562")
print("  Genre  (10 classes) : Top-1 78.5% | Top-5 98.7% | Macro F1 0.759")
print("  Artist (23 classes) : Top-1 78.0% | Top-5 94.8% | Macro F1 0.755")
print()
print("  Artist accuracy (78%) exceeded initial estimate (65-70%),")
print("  which significantly strengthens the pseudo-labeling proposal:")
print("  at 80% confidence threshold, pseudo label noise is substantially")
print("  lower than originally projected.")
```

```
==================================================================
  ARCHITECTURE DECISION SUMMARY
==================================================================

  Full 3-task multi-task         [REJECTED]
  Reason: Artist coverage only 23.4% → ~7/32 labeled per batch → backbone bias

  Intersection only              [REJECTED]
  Reason: 11 style classes missing entirely → 355x class imbalance

  Balanced sampling              [REJECTED]
  Reason: Artist head still only learns from its own images → backbone bias persists

  Style+Genre multi-task         [CHOSEN ✅]
  Reason: 99.5% overlap → ~32 and ~26 labeled per batch → no sparsity

  Separate artist model          [CHOSEN ✅]
  Reason: 100% dense labels every batch → backbone optimised purely for artist discrim
ination

==================================================================
  FINAL RESULTS
==================================================================
  Style  (27 classes) : Top-1 60.0% | Top-5 94.2% | Macro F1 0.562
  Genre  (10 classes) : Top-1 78.5% | Top-5 98.7% | Macro F1 0.759
  Artist (23 classes) : Top-1 78.0% | Top-5 94.8% | Macro F1 0.755

  Artist accuracy (78%) exceeded initial estimate (65-70%),
  which significantly strengthens the pseudo-labeling proposal:
  at 80% confidence threshold, pseudo label noise is substantially
  lower than originally projected.
```